

EE342 Numerical Methods for Electrical Engineering Project 1 Report

Gaussian Elimination with Partial Pivoting

Boran Aybak Kılıç
Boğaziçi University - Electrical and Electronics Engineering

Contents

1	Introduction	2
2	Problem Definition	2
3	Key Concepts from Numerical Methods	2
4	Program Structure	3
5	The Case of High Condition Numbers:	5
6	Comparison Between Manual and AI-Assisted Approaches	5
6.1	Development Time & Ease of Implementation	6
6.2	Code Structure Differences	6
6.3	Feature Comparison	6
6.4	Result Quality & Performance	7
6.5	Key Observations	7
7	Conclusions	8
8	References	8

1 Introduction

This project implements Gaussian Elimination with partial pivoting to solve the linear system $Ax = b$. The solution involves forward elimination, partial pivoting, backward substitution, and for 2x2 matrices, the computation of condition numbers in both 1-norm and infinity-norm. The project is implemented in C++ using dynamic memory allocation, in compliance with the course requirements.

2 Problem Definition

Given a square matrix $A \in \mathbb{R}^{n \times n}$ and a vector $b \in \mathbb{R}^n$, the goal is to solve for x in the linear system $Ax = b$.

Key requirements:

- Read matrix A and vector b from text files.
- Use dynamic memory allocation.
- Implement Gaussian elimination with partial pivoting.
- Detect singular matrices and handle errors gracefully.
- Calculate and display condition numbers for 2×2 matrices.

3 Key Concepts from Numerical Methods

- **Dynamic Memory Management:** Using pointers to dynamically allocate matrices and vectors of arbitrary size.
- **Partial Pivoting:** Selecting the largest available pivot element in each step to improve numerical stability.
- **Gaussian Elimination:** Converting the matrix into an upper triangular form through row operations.
- **Backward Substitution:** Solving the resulting triangular system from bottom to top.
- **Condition Number:** Estimating the sensitivity of the solution to small changes in input for 2×2 systems.

- **Floating Point Precision:** Custom rounding function to manage numerical errors and define an acceptable precision.
- **Error Handling:** Detecting singular matrices and inconsistent dimensions during file reading.

4 Program Structure

The program consists of several key modules:

- **File Reading:** Functions `get_size_of_A_and_b`, `construct_A`, and `construct_b` read and process input files. `construct_A` creates a 2-D array of N pointers pointing to a set of N other pointers with the code snippet:

```

1 // Declare memory block of size M
2 auto** A = new float*[M]; // Set of M pointers
   pointing to other pointers
3
4 for (int i = 0; i < M; i++) {
5     // Declare a memory block of size N
6     A[i] = new float[N]; // Each M pointer points to
   a pointer of size N
7 }
8

```

This structure is necessary since pointers used in dynamic memory only points to the address of the first element of an array. If we need a multidimensional array, we need to point to each first element of the corresponding row.

- **Gaussian Elimination:** Function `gaussian_elimination` performs forward elimination and calls `partial_pivoting` internally to ensure numerical stability with choosing the largest element as the pivot. Since `partial_pivoting` function, swaps corresponding elements of b along with the rows of A , we do not need to keep the indexes of swapped rows since the solution x remains unchanged with this approach. These function are defined as void functions and global parameter A and b are passed by reference meaning that changes made locally inside the function are changed globally as well.
- **Backward Substitution:**
Function `backward_substitution` computes the final solution vector and returns x as a dynamic array.

- **Condition Number Calculation:**

Function `calculate_condition_number` computes the 1-norm and infinity-norm condition numbers for 2×2 matrices. Inverse matrix is calculated easily using basic algebra. Calculating inverses gets exponentially harder as the size of A increases. We only call this function if $N = 2$.

- **Output Formatting:**

Functions `print_problem` and `write_and_print_solution` print matrices and results in a human-readable format. I have also created two functions, as an enhancement, to format the problem statement $Ax = b$ in a visual way to the console. I have taken special care to carefully adjust matrix lines and numbers vertically. I have used `std::setprecision(display_digits)` to print out only a predetermined number of digits to the console. A parameter called *display_precision* is passed as an argument to this function which determines how many digits are going to be displayed. Also, a custom rounding function is created using built-in function `std::round` to round a given number with a desired precision. This function allows user to neglect small errors stemming from floating point arithmetic and finite machine precision. For example, in Fig. 1, we can see that lower triangular part of matrix A after Gaussian elimination is not exactly zero but a very small finite value for *display_precision*= 0.000000000001. This minor discrepancy is removed if the display precision is increased to *display_precision*= 0.00001.

```
Calculating the solution x for the linear system:
[ 3.14000010490 1.59000003338 2.65000009537 3.57999992371 ] [ 5.09999990463 ]
[ 9.78999996185 3.23000001907 8.46000003815 2.64000010490 ] [ 5.82000017166 ]
[ 3.38000011444 3.26999998093 9.50000000000 2.88000011444 ] x = [ 0.97000002861 ]
[ 4.19000005722 7.15999984741 9.39000034332 9.36999988556 ] [ 4.94000005722 ]
Implementing back-substitution on upper triangular matrix:
[ 9.78999996185 3.23000001907 8.46000003815 2.64000010490 ] [ 5.82000017166 ]
[ 0.00000004149 5.7775933472 5.76922369003 8.24011230469 ] [ 2.4491122322 ]
[ 0.00000010097 -0.00000003201 4.42746496281 -1.10473322868 ] x = [ -1.95278918743 ]
[ 0.00000001700 0.00000001132 -0.00000002712 1.78923523426 ] [ 2.72649168968 ]
The solution x is:
[ 0.79327619076 ]
[ -1.68866527081 ]
x = [ -0.06083903462 ]
[ 1.52383077145 ]
```

(a) Solution of problem at instruction for *display_precision*= 0.000000000001

```
[ 3.14000 1.59000 2.65000 3.58000 ] [ 5.10000 ]
[ 9.79000 3.23000 8.46000 2.64000 ] [ 5.82000 ]
[ 3.38000 3.27000 9.50000 2.88000 ] x = [ 0.97000 ]
[ 4.19000 7.16000 9.39000 9.37000 ] [ 4.94000 ]
Implementing back-substitution on upper triangular matrix:
[ 9.79000 3.23000 8.46000 2.64000 ] [ 5.82000 ]
[ 0.00000 5.77760 5.76922 8.24011 ] [ 2.44911 ]
[ 0.00000 0.00000 4.42746 -1.10473 ] x = [ -1.95279 ]
[ 0.00000 0.00000 0.00000 1.78924 ] [ 2.72649 ]
The solution x is:
[ 0.79328 ]
[ -1.68867 ]
x = [ -0.06084 ]
[ 1.52383 ]
```

(b) Solution of problem at instructions for *display_precision*= 0.00001

Figure 1: Comparison of solution displays for different precisions

5 The Case of High Condition Numbers:

As instructed, we will now show how high condition numbers can lead to issues, consider the following example. Consider the following example for matrix

$$A = \begin{bmatrix} 1.000 & 1.000 \\ 1.000 & 1.001 \end{bmatrix}$$

and vectors:

$$b_1 = \begin{bmatrix} 2.000 \\ 2.000 \end{bmatrix} \quad b_2 = \begin{bmatrix} 2.000 \\ 2.001 \end{bmatrix}$$

(a) Solution for $b_1 = [2.000 \ 2.000]$ (b) Solution for $b_1 = [2.000 \ 2.001]$

Figure 2: Comparison of solutions for different right-hand side vectors

As we can see from the solutions provided by the program in Fig. 2:

- For b_1 , the solution x is approximately $x = [2.000 \ 0.000]'$.
- However, for b_2 , the solution x is approximately $x = [1.000 \ 1.000]'$.

Even for a very small change in the input vector b , x deviates significantly, showing the instability of the solution for nearly singular matrices. As can be seen from the console output 2, condition number for A matrix is 4003.81 which is significantly large, indicating ill-conditioning. Thus, such a dramatic change in solution from b_1 to b_2 should not be surprising.

6 Comparison Between Manual and AI-Assisted Approaches

The implementation of Gaussian elimination with partial pivoting was developed through two distinct approaches: manual coding and AI-assisted development. Key differences and observations are outlined below:

6.1 Development Time & Ease of Implementation

- **Manual Coding:** Required approximately 18-20 hours of development time. Significant effort was spent on:
 - Debugging dynamic memory management
 - Designing the custom matrix printer
 - Implementing partial pivoting as a separate function
- **AI-Assisted Coding:** Completed in 30 minutes using ChatGPT 4.0. The AI:
 - Generated boilerplate code for matrix loading
 - Suggested optimized elimination logic
 - Required manual correction for course-specific requirements

6.2 Code Structure Differences

- **Singularity Handling:**
 - Manual: Uses explicit `check_singularity_flag` passed by reference
 - AI: Returns boolean directly from `gaussianElimination`
- **Partial Pivoting Implementation:**
 - Manual: Separate `partial_pivoting` function
 - AI: Integrated within `gaussianElimination` but it still has a separate `swapRows` function.
- **Memory Management:**
 - Manual: Two-step matrix dimension verification in main scope
 - AI: Single-pass loading with row swapping with `loadMatrix` terminating prematurely if a dimension error occurs.

6.3 Feature Comparison

- **Custom Output Formatting:**
 - Manual: Implements sophisticated Unicode box-drawing characters in `print_problem`. Prints out the starting problem and problem after Gaussian elimination, i.e., $Ux = Mb$.

- AI: Uses basic console output without formatting.
- **Precision Handling:**
 - Manual: Custom `round_to` function is created with *display_precision* parameter. Printing functions also take *display_precision* as a parameter and displays the results up to a significant digit.
 - AI: Relies on default floating-point comparisons.
- **Condition Number Calculation:**
 - Manual: Combined 1-norm and inf-norm in single function.
 - AI: Separates norm calculations explicitly.

6.4 Result Quality & Performance

- **Numerical Accuracy:** Both implementations achieve identical results within machine precision.
- **Error Handling:** Manual version provides more detailed error messages.
- **Memory Safety:** AI code uses safer pointer management with:
 - Contiguous memory allocation
 - Early returns on failure
- **Output Quality:** Manual version's formatted printing (Fig. 3) provides superior readability

6.5 Key Observations

- The AI-generated code demonstrated more compact elimination logic but lacked custom formatting features
- Manual implementation showed better adherence to course-specific requirements
- AI assistance significantly reduced coding time but required domain knowledge for:
 - Partial pivoting implementation
 - Condition number calculation

- Precision handling
- The manual approach resulted in more maintainable code through:
 - Modular function design
 - Detailed code comments
 - Consistent variable naming

```
Solution x:
0.793276
-1.68867
-0.060839
1.52383
```

(a) Ai console output

```
Calculating the solution x for the linear system:
[ 3.14000010490  1.59000003338  2.65000009537  3.57999992371 ] [ 5.89999998463 ]
[ 9.78999996185  3.23000001907  8.46000003815  2.64000010490 ] [ 5.82000017166 ]
[ 3.38000011444  3.26999998093  9.50000000000  2.88000011444 ] x = [ 0.97000002861 ]
[ 4.19000005722  7.15999984741  9.39000034332  9.36999988556 ] [ 4.94000005722 ]

Implementing back-substitution on upper triangular matrix:
[ 9.78999996185  3.23000001907  8.46000003815  2.64000010490 ] [ 5.82000017166 ]
[ -0.00000004149  5.77759933472  5.76922369003  8.24011230469 ] [ 2.44911122322 ]
[ -0.00000010097  -0.00000003201  4.42746496201  -1.10473322868 ] x = [ -1.95278918743 ]
[ 0.00000001700  0.00000001132  -0.00000002712  1.78923523426 ] [ 2.72649168968 ]

The solution x is:
[ 0.79327619076 ]
[ -1.68866527081 ]
x = [ -0.06083903462 ]
[ 1.52383077145 ]
```

(b) Self-made console output

Figure 3: Comparison of console outputs for self-made and ai-generated codes

7 Conclusions

Through this project, fundamental numerical methods concepts such as Gaussian elimination, matrix manipulation, condition number sensitivity, and floating-point error management were reinforced. Manual coding enhanced algorithmic thinking and debugging skills, whereas AI-assisted coding improved productivity but at the cost of deeper understanding.

8 References

- EE342 Numerical Methods Lecture Notes, Boğaziçi University.
- <https://cplusplus.com/doc/tutorial/>
- http://cs.bilkent.edu.tr/~saksoy/courses/cs201-Fall2007/src/command_line_arguments.pdf